

e-PG Pathshala
Subject: Computer Science
Paper: Cloud Computing
Module 25: Virtualization IV
Module No: CS/CC/25
Quadrant 1— e-text

1. Introduction

In this module we complete our discussion on virtualization. As we have seen already virtualization is a necessary mechanism in a data center providing cloud-based services. In the last module we've explained the fundamentals of virtualization, the objective of this module is to provide an overview on methods of virtualization, especially the procedure to implement CPU virtualization. We conclude this module, as well as our discussion on virtualization with an overview of a free and open source tool for virtualization named is Xen, and its architecture as case study.

2. Learning Outcome

At the end of this module, students will be able to:

1. Understand the general methods of implementing virtualization.
2. Understand how emulation is used for virtualization.
3. Get knowledge about the method of binary translation.
4. Learn the importance of paravirtualization.
5. Get knowledge about the hardware-assisted virtualization.
6. Learn the basic architecture of Xen, an open-source hypervisor.

3. Methods of Virtualization

As mentioned in the earlier module, there are three generic techniques that can be used for virtualization. These are:

1. Emulation
2. Trap and Emulate.
3. Binary Translation

Before explaining the methods in practice for virtualization, it is important to understand these techniques.

3.1. Emulation

Emulation is the process where the virtualizing software mimics that portion of hardware, which is provided to the guest operating system in the virtual machine. The presented emulated hardware is independent of the underlying physical hardware. Emulation provides VM portability and wide range of hardware compatibility, which means the possibility of executing any virtual machine on any hardware, as the guest operating system interacts only with the emulated hardware.

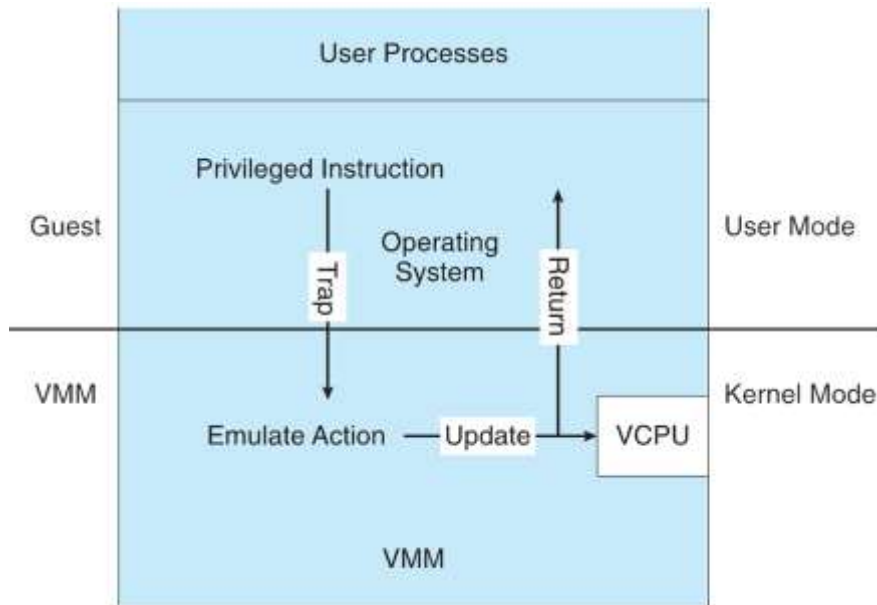
In an emulated environment, both the application and guest operating system in virtual machines run in the user mode of base operating system. In simple terms, the behavior of the hardware is produced by a software program. Emulation process involves only those hardware components so that user or virtual machines does not understand the underlying environment. Only CPU & memory are sufficient for basic level of emulation.

Typically, emulation is implemented using interpretation. The emulator component takes each and every instruction of user mode and translates to equivalent instruction suitable according to the underlying hardware. This process is also termed as interpretation. This means that the guest OS remains completely unaware of the virtualization. Also, in interpretation, each and every instruction issued by a VM is trapped in the VMM and interpreted for execution in the hardware. Goes without saying that computationally it is a very expensive method. However, in some cases,, it is needed to use an interpretation technique. However, due to the huge disadvantage of performance, emulation using interpretation is hardly used in virtualization.

3.2. Trap-and-Emulate

Trap and emulate is a technique that takes the basic of the emulation but improves performance by using interpretation selectively. In this method also, both the user applications and guest operating system of virtual machines run in the user mode and the hypervisor runs in the privileged mode. However, here the application runs the instructions natively on the hardware. Only when a privileged instruction is to be executed in virtual user mode, a trap to the virtual kernel mode occurs. This causes a trap to the VMM in turn. The hypervisor gains control of the execution. All the special instructions that need to be managed, are managed by the VMM and not by the guest OS.

When hypervisor traps, it executes the necessary equivalent operations in the underlying instruction set architecture and returns control to guest in user mode. User mode code in guest runs at normal speed. There is no change from running in non-virtualized environment. But kernel mode privileged codes run slower due to trap-and-emulate. The mechanism works fine, except that the CPU is slower in kernel mode. Unfortunately, it becomes a performance issue when there are several guests and each guest needs to trap to VMM for all privileged mode instructions.



Courtesy: [4]

Figure 25.1: Trap and Emulate

The privileged instruction from the guest operating system is trapped by the hypervisor and emulated. This emulated instruction is then passed on to the kernel. On return the privileged instruction from the kernel is emulated and returned to the user mode where the operating system of virtual machine is running.

There are issues prevailing in Trap and Emulate methodology. Not all ISAs can be emulated using trap-and-emulate method. Those that do not follow Popek-Goldberg theorem cannot be emulated. Sensitive instructions are not Privileged instructions. Let us consider the example of Intel x86 popf instruction. The CPU flags are loaded from the contents of stack. In privileged mode, all flags are replaced whereas in user mode only some flags are replaced, hence no trap is generated.

The following is an example to understand why trap and emulate does not work.

Let us take the example of the popf instruction. This instruction loads a set of flags from the stack into a register and can be executed in both the user mode and privileged mode. When executed in privileged mode, popf loads all flags, which include a mix of ALU flags and system flags. In this mode, both the set of flags get modified, as per requirement, since the OS has the permission to make necessary changes in the system flags. However, when the same popf gets executed in user mode, the processor simply does not allow modification of the system flags even if the situation demands it. But, no error or exception is generated due to this. This is what is meant by 'going totally unnoticed' by the underlying OS.

This works fine in a non-virtualized environment. Now, let us see why this would cause a problem in a virtualized environment using a trap and emulate kind of mechanism. The guest user mode runs the popf instruction without altering the system flags and this is just a normal action. What happens when the guest OS executes the popf instruction? It is 'expected' to modify the system flags as per requirement. But since this itself is running in the user mode, it will not be allowed to modify the system flags! However, a question here is that the same is applicable for all other privileged instructions in this scenario yet there were no problems with them. The explanation to this question is as follows: Since, in all other privileged

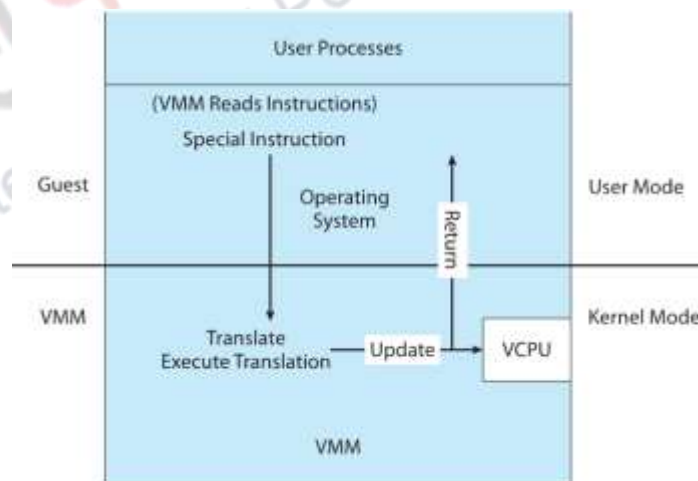
instructions when the guest OS attempts to execute a privileged instruction, the processor will detect this 'unnatural' behaviour and will wake up the VMM. However, since processor is supposed to allow the execution of popf in user mode as well, it will not raise the VMM when the guest OS attempts to execute the instruction. It just does not allow the guest OS to modify the system flags since it is expecting the guest OS to 'behave' like an application and not as an OS. As a result, the outcome of this execution is not the same as the expected outcome of this instruction.

3.3. Binary Translation

The issue of popf and other such instructions that occur in the trap and emulate method can be taken care of if the VMM gets more proactive. This additional activity of VMM is the Binary Translation. Although the concept of binary translation is simple, the complexity lies in implementation of binary translation.

If guest vCPU is in user mode, guest can run instructions natively, whereas if guest vCPU is in kernel mode, VMM checks every instruction (and does not wait for a trap!). Non-sensitive instructions run normally but sensitive instructions are translated appropriately. Performance of this method is worse than trap-and-emulate since all codes of the guest kernel is inspected by the VMM. However, many optimizations have been proposed. Let us look at such an optimization.

One such available optimization for improving the performance is caching. In caching technique, a block of instructions are translated once and stored in cache. When guest executes the code with sensitive instructions, the cached translation is first checked. If instruction is available in cache as translated instruction, then no more translation is necessary. Translation takes place otherwise. This improves the efficiency of Binary Translation.



Courtesy: [4]

Figure 25.2: Binary Translation

4. CPU Virtualization

With the above methods being available, it is time to see what the cloud industry has actually used to virtualize various hardware environments. As expected, the industry responded to the need of virtualization and came up with multiple solutions, some of

which took advantage of the methods discussed above, yet others came up with unconventional solutions. Over all, there are four methods in CPU virtualization:

1. Emulation with Interpretation.
2. Full virtualization with dynamic binary translation.
3. Para virtualization.
4. Hardware-assisted virtualization.

4.1. Emulation with Interpretation

This is the virtualization at the ISA level as we have discussed in our earlier discussion on the implementation of hardware virtualization. Although they are called virtualization, this mechanism is largely used when a program that is written for one ISA needs to run on a different ISA. For example, one may need to run an android application written for Android platform in a Windows-based machine. An appropriate emulator may be run on a Windows box. Some of the very popular platforms available are Bochs and QEMU.

Of these two, QEMU, besides being an emulator, even has a virtualizer, which attempts to bypass the emulator and run some of the x86 codes natively.

4.2. Full Virtualization with Dynamic Binary Translation

The requirement in a virtualized environment is to run an existing operating system along with applications in an isolated virtual machine. We should be able to run many such VMs in such a way that these do not affect each other's performance running on the same hardware. In an x86 architecture, the guest OSs and VMs are not executed in ring 0. There are two issues out of this action. The first is that these guest OSs should be supervised by a host OS running in ring 0. Hence VMM is run in ring 0. Where do we run the guest OS? These must run deprived in either ring 1 or 2. Typically the application is run in ring 3 and guest OS in ring 1. This methodology is termed as *full virtualization*.

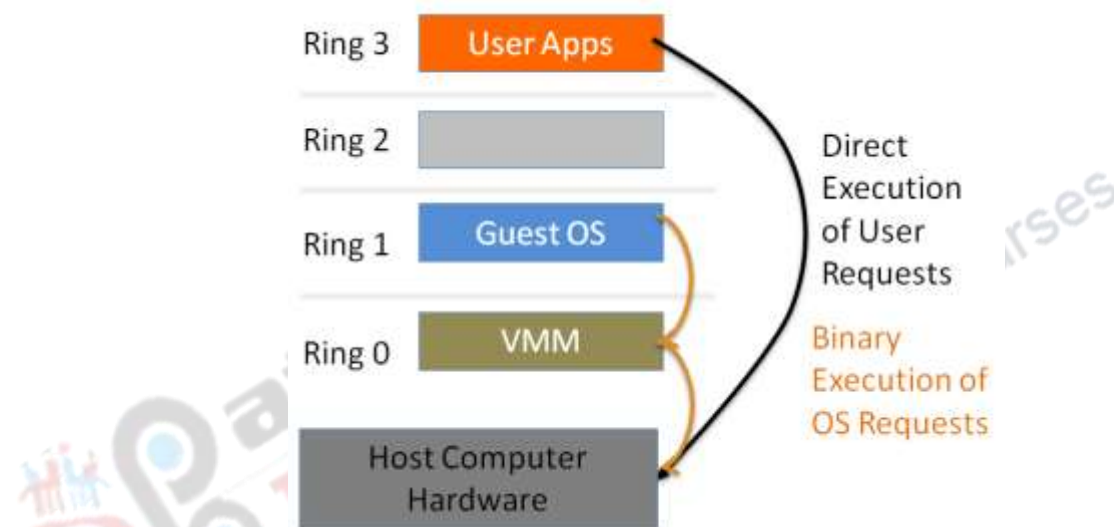
However, depriving an OS comes with an associated cost. Since the guest operating systems are written for execution on top of the hardware (ring 0), many instructions may be unsafe or even potentially harmful to be run in user mode. Simple traps for all these 'sensitive' instructions may not work (as per Popek-Goldberg). To ensure safety of the virtualized environment, all those instructions that can cause problems must be intercepted and rewritten, if required. Hence complete binary translation of the kernel code of the guest OSs is required to ensure the safety of the processor and the machine while allowing the user code to run natively on the hardware. Further, a guest OS could easily determine that it is not running at privilege level 0. Since this is not desirable, the VMM must take an appropriate action. Another problem of depriving the OS is that, as normal OS, the guest OS program expects to enjoy unrestrained access to the full memory but as a mere application, running in a higher ring, it cannot enjoy the same privilege. Hence the VMM must make way for ensuring that this is taken care of.

This method is called the full virtualization with binary translation. In this virtualization one or more guest operating systems of virtual machines share hardware resources from the host system. The presence of the hypervisor beneath is not known to the guests.

However, the issue that restricts full virtualization with binary translation is the performance. Translation takes time and translating all the kernel codes of the guest OS is expensive in terms of performance. This problem is resolved by using dynamic binary translation.

In dynamic binary translation, a block of code is used. These blocks may or may not have critical instructions. For each block, dynamic BT translates critical instructions, if any, into some privilege instructions, which will trap to VMM for further emulation. Full virtualization technology uses and exploits dynamic binary translation.

The execution of instructions in full virtualization using dynamic binary translation is explained in the Figure 25.4.



Courtesy: [2]

Figure 25.3: Full Virtualization with Dynamic BT

The virtual machine monitor or hypervisor executes in ring 0, guest OS in ring 1 and application in ring 3. The black arrow shows the direct execution of user request on the hardware. The orange line shows the binary translation of the OS code. The VMM translates the guest OS's instructions from ring 1 and passes to hardware for execution.

Although this methodology provides performance, there are shortcomings in this methodology. Binary translation does not work for certain cases where the guest OS may be using self-modifying or self-referencing codes. Real-time systems also cannot be virtualized since such systems cannot tolerate the delays caused by the translation.

For the above-mentioned scenario, other methods of virtualization are to be adopted.

4.3. Para Virtualization

The problem of full virtualization is that the guest OS is unaware of the fact that it has been diprivileged and hence its behaviour continues to be the same. In para virtualization, the guest OS is modified or patched for virtualization. Hypervisor sits as the base OS or in ring 0 in case of x86 and guest OS resides on top of VMM. Here, since the Guest OS is aware that it is running above VMM rather than on top of the physical machine, many problems of full virtualization is taken care of. The

modified kernel of the guest OS is able to communicate with the underlying hypervisor via special calls. These special calls are provided by specific APIs depending on the hypervisor employed. These special calls are equivalent to system calls generated by an application to a non virtualized OS. Xen Hypervisor is an example that uses paravirtualization technology.

The Guest OS is modified and thus run kernel-level operations at Ring 1. The guest OS is now fully aware of how to process both privileged and sensitive instructions. Hence the necessity for translation of instructions is not present any more. Guest OS uses a specialized call, called “hypercall” to talk to the VMM. VMM executes the privileged instructions. Thus VMM is responsible for handling the virtualization requests and putting them to the hardware.

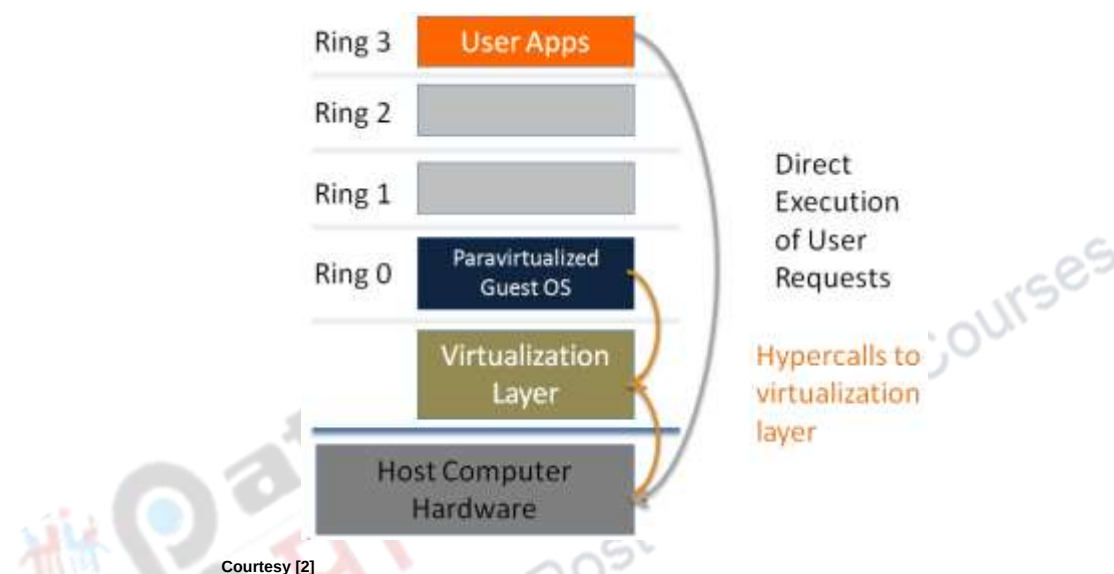


Figure 25.4: Paravirtualization

Paravirtualization mode in the Figure 25.4 shows the hypercalls using orange lines, while the grey line shows the native execution of the user process as usual. Hypercalls are passed onto the hypervisor in the virtualization layer and VMM passes the calls to hardware after processing. The issue here is the fact that the guest OS must be modified. This causes problems in OS maintainability and supportability. Also, since the guest OS and hypervisor are tightly coupled, compatibility problems arise. Each time the hypervisor is updated, guest OS must also be recompiled.

4.4. Hardware-Assisted Virtualization

Legacy processors are not designed for virtualization. Hence we observed that whatever the methods that may be applied for implementing virtualization, each has its own problems. However, if the processors are made virtualization-aware, the VMM design will be more efficient and simple. Many issues mentioned in the earlier sub-section can be easily taken care of with such a processor.

This is the reason why hardware vendors rapidly embraced virtualization and developed new features to simplify virtualization techniques. The two giants in the hardware arena, Intel and AMD came up with designs of new CPU execution mode that allows VMM to run in a new root mode below ring 0. This is the way to handle the privileged mode. In this new design, both privileged and sensitive calls automatically trap to the hypervisor. Hence, in this new design, there remains

no need for either binary translation or paravirtualization. Now, the latest x86's meet Popek & Goldberg requirements, hence they can be virtualized without any complexities. Examples of this new design are Intel VT-x (2005) and AMD-V (2006).

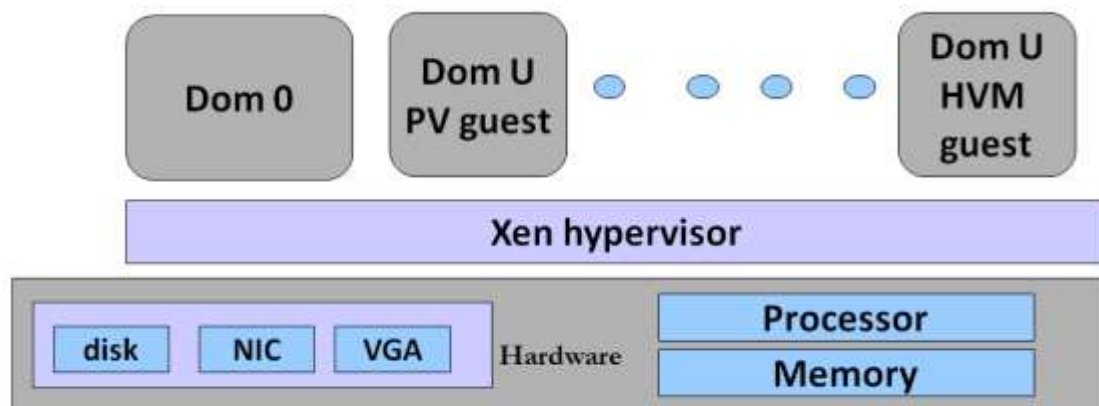
Intel VT-x have two modes of operations: VMX root and VMX non-root. While “VMX root” mode operation executes the hypervisor / VMM in the ring 0, “VMX non-root” mode operation executes the guest OS, also in ring 0, thereby removing the need to deprive the guest OS. Both the modes support all privilege rings and are identical. Unmodified guest OS runs in ring 0 in non-root mode and traps instructions to root mode. The privileged and sensitive calls automatically trap to the hypervisor. VMM controls the execution of the guest OS.

While it is possible to run unmodified guest OS in the above set up thereby allowing execution of legacy/unmodified operating system, the challenge in hardware assisted virtualization is that an unmodified OS cannot take the advantage of virtualization. The OS is unaware of whether it is running in a virtualized environment or conventional environment. Partial implementation of paravirtualization may solve this problem.

Now that we have seen how the four types are used in four different environments to implement virtualization, let us look into one such implementation, i.e., paravirtualization by Xen.

5. Case Study: Xen

Xen is a free and open source hypervisor widely employed in virtualization environment. The name of this software, Xen, is derived from **neXtgENeration**. Xen Hypervisor adopts paravirtualization and Hardware-assisted full virtualization. Figure 25.6 explains the architecture of Xen.



Courtesy [5]

Figure 25.6: Xen Architecture

Xen hypervisor resides on top of the hardware and is responsible for handling all the low-level functionalities. The hardware comprises storage in the form of disks, network interface card, graphics-display adapters, processor and main memory. The whole environment is divided into domains or virtual machines in which the guest OSs reside and these are called Doms or domains. Dom 0, which hosts the most important operating system, is a privileged domain and is responsible for the creation of other new domains. The OS in Dom 0 is a modified privileged guest operating

system inbuilt with the hypervisor. Domain 0 is designed to access hardware directly and manage devices.

All other virtual machines are Dom U which is the guest operating system. These also interact with hardware through the hypervisor. The physical hardware is directly inaccessible for the guest OSs and VM. Linux, Solaris, FreeBSD, UNIX are categorized as paravirtualized VMs whereas Windows VMs are categorized as full virtualized VMs – DomU HVM guests.

Xen is widely employed as a hypervisor with KVM – Kernel Virtual Machine as an alternate free and open source hypervisor. Xen is widely adopted in cloud and virtualization environment. Some of the enterprises supporting Xen are Sun Microsystems, Hewlett-Packard, Novell, Red Hat, Intel, Advanced Micro Devices, Voltaire, IBM.

6. Summary

In this module, we first explored various ways of implementing virtualization. We observed that emulation technique is the basis for virtualization, which uses interpretation and translation approaches. In this module, the generic methods of virtualization are discussed first with greater emphasis on the practical implementation of virtualization as adopted by the industry. Among these, full virtualization with dynamic binary translation and paravirtualization are the most used methods. However certain issues still persists, hence at certain instance hardware assisted virtualization. In hardware assisted virtualization, the VT-x approach is followed in VM-x root and non-root mode. As a case study, we briefly looked at Xen, an open-source free hypervisor based on paravirtualization.

In the next two modules we investigate the next topic that is also very important in cloud environment, called web services.

References

1. Smith, J. E., and Ravi Nair, "Virtual Machines: Architectures, Implementations and Applications", Morgan Kauffmann, 2004.
2. Marshall, David. "Understanding Full Virtualization, Paravirtualization, and Hardware Assist", VMWare White Paper, 2007.
3. Figueiredo, Renato, and Peter A. Dinda, "Guest Editors' Introduction: Resource Virtualization Renaissance", Computer 5: 28-31, 2005.
4. Greg Gagne, Peter B. Galvin, A. Silberschatz, "Operating System Concepts", 9th Edition, John Wiley & Sons, 2012.
5. Hwang, Kai, Jack Dongarra, and Geoffrey C. Fox, "Distributed and Cloud Computing: From Parallel Processing to the Internet of Things", Morgan Kaufmann, 2013.